

ShopKart Data Engineering Platform

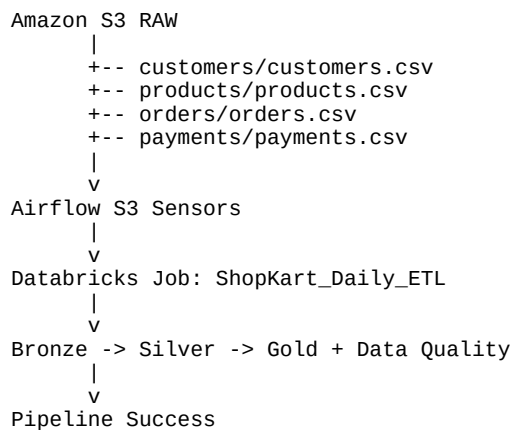
Airflow Phase Documentation — AWS S3 + Databricks Orchestration

Purpose: GitHub-ready project documentation for the Airflow orchestration phase.

1. Airflow Phase Overview

Apache Airflow is used as the orchestration layer for the ShopKart pipeline. The DAG checks whether the required raw files are available in Amazon S3 and, after all four files are available, triggers the Databricks ETL job. The final task records successful completion.

Pipeline Flow



2. DAG Configuration

Parameter	Value
DAG ID	shopkart_daily_pipeline
Schedule	@daily
Catchup	False
AWS Connection	aws_shopkart
S3 Bucket	shopkart-data-2026
Raw Prefix	raw
Databricks Connection	databricks_shopkart
Databricks Job ID	478726098061363
Retries	2
Retry Delay	2 minutes

3. Tasks in the DAG

start_pipeline — PythonOperator: Logs pipeline start and configuration.

check_customers_file — S3KeySensor: Waits for raw/customers/customers.csv.

check_products_file — S3KeySensor: Waits for raw/products/products.csv.

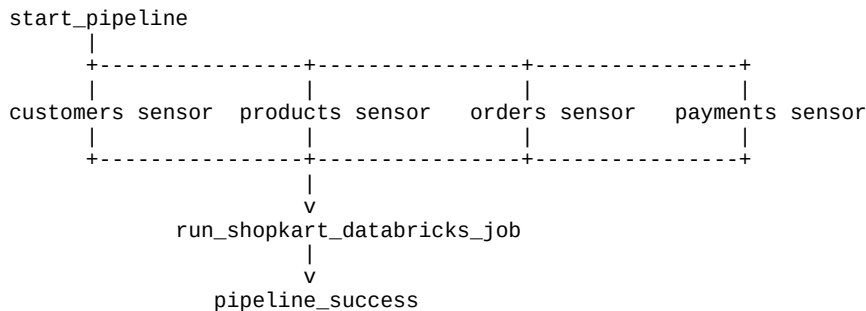
check_orders_file — S3KeySensor: Waits for raw/orders/orders.csv.

check_payments_file — S3KeySensor: Waits for raw/payments/payments.csv.

run_shopkart_databricks_job — DatabricksRunNowOperator: Triggers the Databricks ETL job.

pipeline_success — PythonOperator: Logs successful pipeline completion.

4. Dependency Design



5. Why S3 Sensors Are Used

Each S3KeySensor verifies that a required source object exists before the ETL job starts. The configured sensors use a 30-second polling interval, a 300-second timeout, and **reschedule** mode. This prevents the Databricks job from starting before the four required inputs are present.

6. Airflow Connections Used

aws_shopkart — Airflow connection used by the S3 sensors to access the ShopKart S3 bucket.

databricks_shopkart — Airflow connection used by DatabricksRunNowOperator to trigger the Databricks job.

Credentials are stored in Airflow connections rather than hard-coded into the DAG. Do not place AWS access keys, Databricks tokens, or passwords in GitHub.

7. Successful Airflow Run

The provided Airflow screenshot shows a successful manual run of **shopkart_daily_pipeline**. The S3 sensors, Databricks task, start task, and success task are shown as successful. The Databricks task ran for approximately 4 minutes 17 seconds, and the complete DAG run duration shown is approximately 4 minutes 23 seconds.

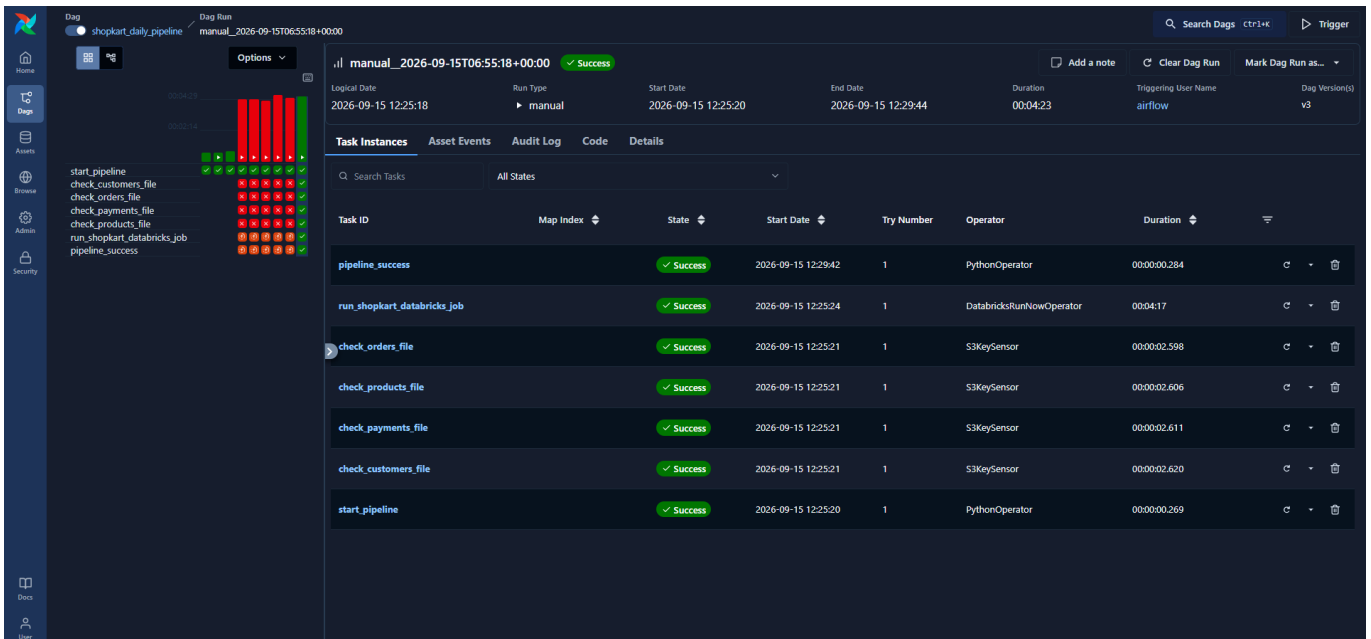


Figure 1 — Successful shopkart_daily_pipeline Airflow run.

8. GitHub Repository Structure for Airflow

```
airflow/  
  dags/  
    shopkart_daily_pipeline.py  
  docker-compose.yaml
```

The main production DAG to upload is **dags/shopkart_daily_pipeline.py**. The Docker Compose file may also be included if it contains no secrets.

9. Files to EXCLUDE from GitHub

Do not upload: .env, logs/, __pycache__/, generated credentials/password files, AWS keys, Databricks tokens, Redshift passwords, or other secrets.

The supplied project contains an .env file, Airflow logs, Python __pycache__ files, and configuration material. These should be excluded from the public repository. The supplied **test_dag.py** also contains hard-coded authentication values, so it should not be uploaded unchanged.

10. Recommended .gitignore

```
# Secrets  
.env  
*.env  
.env.*  
*.pem  
*.key  
credentials.json  
secrets.json  
  
# Python  
__pycache__/  
*.pyc  
.venv/  
venv/  
# Airflow
```

```
logs/  
  
# IDE / OS  
.vscode/  
.idea/  
.DS_Store  
Thumbs.db  
  
# Local archives / large files  
*.zip  
*.csv
```

11. Main Airflow DAG — Source Code

The following is the supplied ShopKart production DAG source used for the successful Airflow run.

```
from datetime import datetime, timedelta

from airflow.sdk import DAG
from airflow.providers.standard.operators.python import PythonOperator
from airflow.providers.amazon.aws.sensors.s3 import S3KeySensor
from airflow.providers.databricks.operators.databricks import DatabricksRunNowOperator

# =====
# SHOPKART CONFIGURATION
# =====

S3_BUCKET = "shopkart-data-2026"

AWS_CONN_ID = "aws_shopkart"

RAW_PREFIX = "raw"

DATABRICKS_CONN_ID = "databricks_shopkart"

DATABRICKS_JOB_ID = 478726098061363

# =====
# FUNCTIONS
# =====

def start_pipeline():

    print("=" * 70)
    print("SHOPKART DAILY DATA ENGINEERING PIPELINE STARTED")
    print("=" * 70)

    print(f"S3 Bucket      : {S3_BUCKET}")
    print(f"AWS Connection   : {AWS_CONN_ID}")
    print(f"Raw Prefix       : {RAW_PREFIX}")
    print(f"Databricks Job ID : {DATABRICKS_JOB_ID}")

def pipeline_success():

    print("=" * 70)
    print("SHOPKART PIPELINE COMPLETED SUCCESSFULLY")
    print("=" * 70)

# =====
# DAG
# =====

with DAG(

    dag_id="shopkart_daily_pipeline",

    start_date=datetime(2026, 9, 14),

    schedule="@daily",

    catchup=False,

    default_args={
        "owner": "shopkart-data-engineering",
        "retries": 2,
        "retry_delay": timedelta(minutes=2),
    },

    tags=[
        "shopkart",
        "aws",
        "s3",
        "databricks",
        "pyspark",
        "delta-lake",
    ],

) as dag:

    # =====
    # START
    # =====

    start = PythonOperator(
        task_id="start_pipeline",
        python_callable=start_pipeline,
    )

    # =====
```

```

# CUSTOMERS
# =====

check_customers = S3KeySensor(
    task_id="check_customers_file",
    bucket_name=S3_BUCKET,
    bucket_key=f"{RAW_PREFIX}/customers/customers.csv",
    aws_conn_id=AWS_CONN_ID,
    poke_interval=30,
    timeout=300,
    mode="reschedule",
)

# =====
# PRODUCTS
# =====

check_products = S3KeySensor(
    task_id="check_products_file",
    bucket_name=S3_BUCKET,
    bucket_key=f"{RAW_PREFIX}/products/products.csv",
    aws_conn_id=AWS_CONN_ID,
    poke_interval=30,
    timeout=300,
    mode="reschedule",
)

# =====
# ORDERS
# =====

check_orders = S3KeySensor(
    task_id="check_orders_file",
    bucket_name=S3_BUCKET,
    bucket_key=f"{RAW_PREFIX}/orders/orders.csv",
    aws_conn_id=AWS_CONN_ID,
    poke_interval=30,
    timeout=300,
    mode="reschedule",
)

# =====
# PAYMENTS
# =====

check_payments = S3KeySensor(
    task_id="check_payments_file",
    bucket_name=S3_BUCKET,
    bucket_key=f"{RAW_PREFIX}/payments/payments.csv",
    aws_conn_id=AWS_CONN_ID,
    poke_interval=30,
    timeout=300,
    mode="reschedule",
)

# =====
# DATABRICKS JOB

```

```
# =====

run_databricks = DatabricksRunNowOperator(

    task_id="run_shopkart_databricks_job",

    databricks_conn_id=DATABRICKS_CONN_ID,

    job_id=DATABRICKS_JOB_ID,

)

# =====
# SUCCESS
# =====

success = PythonOperator(

    task_id="pipeline_success",

    python_callable=pipeline_success,

)

# =====
# DEPENDENCIES
# =====

start >> [
    check_customers,
    check_products,
    check_orders,
    check_payments,
]

[
    check_customers,
    check_products,
    check_orders,
    check_payments,
] >> run_databricks >> success
```

12. Git Commands

```
git status
git add airflow/
git commit -m "Add Airflow orchestration pipeline"
git push
```

13. Interview Explanation

"I built an Airflow DAG for the ShopKart data pipeline. The DAG runs daily, checks the availability of four raw source files in Amazon S3 using S3KeySensor, and only after all inputs are available it triggers a Databricks ETL job using DatabricksRunNowOperator. The Databricks job processes the data through Bronze, Silver and Gold layers with data-quality checks. I also configured retries and used reschedule mode for the S3 sensors. The DAG was tested successfully in the Airflow UI."

14. AWS Phase Covered by This Airflow Layer

This Airflow phase demonstrates the integration between Airflow and AWS S3. S3 is the raw-data landing area, Airflow validates object availability, and the orchestration layer then triggers Databricks for processing. The actual AWS resources remain in AWS; GitHub stores the code and documentation that reproduce or explain the pipeline.

Generated from the supplied ShopKart project files and the provided Airflow successful-run screenshot. This document is intended as project documentation; the source code files should remain separately stored in the GitHub repository.